Assignment 2 solution

1. 【考点：进程与线程】在多对一（Many-to-One，即用户级线程）模型下，关于线程的调度与阻塞机制，下列说法正确的是?

   A. 当其中一个用户级线程发起阻塞式系统调用（如 `read()`）时，整个进程将被内核阻塞
   B. 内核可以感知到各个用户级线程的存在，并能将它们调度到不同的 CPU 核心上并行执行
   C. 用户级线程的上下文切换必须从用户态陷入内核态由内核完成，因此切换开销较大
   D. 一个用户级线程占用的 CPU 时间片耗尽后，内核会自动调度同进程内的另一个用户级线程执行

   【答案】：A
   【解析】：
   A选项正确：在多对一模型中，内核只认识进程，不感知用户级线程。如果一个用户级线程发起阻塞式系统调用，内核会将整个进程置于阻塞状态，导致该进程内的所有其他用户级线程均无法运行
   B选项错误：内核感知不到用户级线程，因此无法将其分配到多核 CPU 上并行
   C选项错误：用户级线程的切换在用户态由线程库完成，无需切换到内核态，开销小
   D选项错误：时间片中断由内核处理，内核只能将 CPU 剥夺并调度其他进程，进程内线程的切换由用户态线程库决定

2. 【考点：进程间通信】关于 UNIX/Linux 操作系统中的信号（Signal）机制，下列说法正确的是?

   A. 信号是一种同步通信机制，发送方必须等待接收方处理完信号后才能继续执行
   B. 进程可以通过捕获并注册自定义处理函数来响应所有类型的信号，包括 SIGKILL 和 SIGSTOP
   C. 信号可以在内核与进程之间、以及进程与进程之间传递，用于通知发生了某种异步事件
   D. 信号机制通过在内存中开辟一块共享缓冲区，主要用于直接传递大块的业务数据

   【答案】：C
   【解析】：
   A选项错误：信号属于异步通知机制，发送方发送信号后无需等待接收方处理即可继续执行
   B选项错误：`SIGKILL`（终止进程）和 `SIGSTOP`（暂停进程）是特例，不能被捕获、忽略或阻塞，只能执行默认动作
   D选项错误：信号仅携带极少量的状态信息（如信号编号），主要用于通知事件发生，不能用于传输大量数据（传输大块数据应使用共享内存或管道）
   C选项正确：信号是内核或进程发出的异步通知信号，用于处理中断事件或异常

3. 【考点：CPU调度与上下文切换】在对称多处理器（SMP）系统中，为了优化 CPU 级缓存（Cache）命中率并减少跨核数据同步开销，操作系统调度器通常采用的策略是?

   A. 处理器亲和性（Processor Affinity）策略，尽量将一个进程/线程固定在同一个物理 CPU 核心上运行
   B. 频繁进行推拉式负载均衡（Push/Pull Load Balancing），确保所有 CPU 核心的队列长度时刻绝对一致
   C. 采用全局单队列调度，每次 CPU 核心空闲时都强制从全局队列头部抢占任务
   D. 屏蔽所有内核级线程，强行改用用户级线程来实现跨核并行

   【答案】：A
   【解析】： 在 SMP 架构中，进程在一个 CPU 核心上运行时，其热点数据和指令会被加载到该核心的 L1/L2 Cache 中。采用处理器亲和性（Processor Affinity）策略，让进程尽量在原核心上续跑，可以极大提高 Cache 命中率。频繁移动进程（如B和C）会导致 Cache 大面积失效（Cache Miss），增加总线同步开销

4. 【考点：CPU调度算法】某单核 CPU 系统采用非抢占式高响应比优先（HRRN）调度算法。在 t = 0 时刻，进程 P_1 到达并立即投入运行，运行时间为 8ms。在 P_1 运行期间，P_2（于 t = 2ms 到达，运行时间为 6ms）和 P_3（于 t = 4ms 到达，运行时间为 2ms）陆续到达。当 P_1 执行完毕（t = 8ms）时，系统选择下一个执行的进程及 P_3 的周转时间分别是？

   A. 选择 P_2；P_3 的周转时间为 8ms
   B. 选择 P_3；P_3 的周转时间为 6ms
   C. 选择 P_3；P_3 的周转时间为 8ms
   D. 选择 P_2；P_3 的周转时间为 6ms

   【答案】：B
   【解析】：
   响应比计算公式：

$$R = \frac{\text{等待时间} + \text{服务时间}}{\text{服务时间}} = 1 + \frac{\text{等待时间}}{\text{服务时间}}$$

   在 t = 8ms 时，P_1 运行结束，评估就绪队列中的 P_2 和 P_3：
   P_2：等待时间 = 8 - 2 = 6ms，服务时间 = 6ms，响应比 R_{P2} = (6 + 6) / 6 = 2.0
   P_3：等待时间 = 8 - 4 = 4ms，服务时间 = 2ms，响应比 R_{P3} = (4 + 2) / 2 = 3.0
   因为 R_{P3} (3.0) > R_{P2} (2.0)，系统选择 P_3 运行
   P_3 于 t = 8ms 开始运行，服务时间 2ms，在 t = 10ms 时完成
   P_3 的周转时间 = 完成时间 - 到达时间 = 10 - 4 = 6ms

5. 【考点：调度机制】关于操作系统的闲逛进程（Idle Process）和 CPU 上下文切换，下列说法正确的是？

   A. 闲逛进程的优先级通常设置为最高，以保证在无其他就绪进程时能够被迅速激活
   B. 当系统中没有其他就绪进程时，CPU 将自动进入关机或死锁状态，不再执行任何指令
   C. 闲逛进程不可以被任何外部硬件中断打断，直到其自愿放弃 CPU 控制权
   D. 闲逛进程的优先级最低，通常执行低功耗指令（如 `HLT`），且永远不会进入阻塞状态

   【答案】：D
   【解析】：
   闲逛进程（Idle）是操作系统内核在系统完全没有其他就绪进程可运行时执行的一个特殊进程
   它的优先级最低（保证任何普通进程就绪都能立即抢占它），且永远处于运行或就绪态，绝不进入阻塞态。在 PID 为 0 的 Idle 循环中通常执行低功耗指令（如 `HLT`）以节省电能。硬件中断（如时钟中断、网卡中断）发生时能随时唤醒 CPU 并退出 Idle 状态

6. 【考点：同步与互斥】硬件指令 `TestAndSet`（TSL 指令）常用于实现进程互斥。下列关于 TSL 指令的描述及应用中，说法正确的是？

   A. TSL 指令是在软件层面由操作系统内核通过逻辑封装保证原子性的
   B. 使用 TSL 指令实现临界区互斥时，未能进入临界区的进程会主动进入阻塞队列，遵循“让权等待”原则
   C. TSL 指令是一个不可中断的硬件原子指令，通过读出指定内存变量的值并将其设置为 True 来实现锁操作
   D. 在单核 CPU 环境下，TSL 指令完全不需要硬件支持即可由用户态程序独立实现

   【答案】：C
   【解析】：
   A、D选项错误：TSL（Test-and-Set Lock）是由硬件电路直接支持的原子指令，在执行过程中读出并修改内存单元的操作不可被打断（多核下还会锁总线）
   B选项错误：基于 TSL 实现的简易锁属于忙等锁（Spinlock / 自旋锁），未抢到锁的进程会在 `while(TSL(&lock))` 中持续循环检查，消耗 CPU 时间，违反了“让权等待”原则
   C选项正确：TSL 原子性地读出原值并写入 True，若原值为 False 则代表成功加锁

7. 【考点：同步与互斥】在 Pthreads 多线程同步编程中，条件变量（Condition Variable）通常必须与互斥锁（Mutex）配合使用。关于 `pthread_cond_wait(&cond, &mutex)` 操作的执行逻辑，下列描述正确的是？

   A. 调用该函数前无需获取互斥锁，函数会自动在内部完成加锁操作
   B. 该函数在将线程放入条件变量等待队列的同时，会自动且原子地释放传入的互斥锁
   C. 当线程被 `pthread_cond_signal` 唤醒时，它会立刻无条件地从 `pthread_cond_wait` 返回并执行下一行代码，无需重新获取互斥锁
   D. 条件变量本身包含一个整数计数器，用于记录当前有多少个资源可用

   【答案】：B
   【解析】：
   A、B选项：调用 `pthread_cond_wait` 前必须先持有互斥锁。进入等待时，为了防止"错过唤醒信号（Lost Wake-up）"竞争条件，该函数会自动且原子地将线程挂起并释放互斥锁（B正确）
   C选项错误：线程被唤醒后，必须重新竞争并重新获取互斥锁，才能从 wait 返回
   D选项错误：条件变量没有内部计数器（信号量 Semaphore 才带计数器），它仅代表一个等待队列，因此必须配合用户定义的条件变量检查（通常置于 `while` 循环中）

8. 【考点：经典同步问题】在"读者-写者"问题的写者优先（Writer-First）实现中，引入了一个额外的互斥信号量（例如 `wsem`）来阻止后续读者进入。若当前有一个读者正在临界区内读取数据，且此时有一个写者在等待队列中，后续到达的新的读者进程将被（ ）

   A. 允许直接进入临界区与前一个读者同时读取，因为读者之间不互斥
   B. 阻塞在 `wsem` 或相关的信号量上，直到写者完成写入后才可能被唤醒
   C. 强行将正在读取的读者踢出临界区，以优先保证写者写入
   D. 触发死锁，导致后续所有读者和写者均无法继续运行

   【答案】：B
   【解析】：
   写者优先的核心逻辑是：只要有写者发出写请求（即使因为当前有读者在读而被迫等待），后续到达的所有读者都必须在门外（信号量上）排队阻塞，不能因为"读者之间不互斥"而继续涌入。只有当正在读的读者退出、等待的写者进区完成写操作后，门外排队的读者才会被唤醒

9. 【考点：死锁】某系统中有 3 种资源 R_1, R_2, R_3，当前有 4 个进程 P_1, P_2, P_3, P_4，它们的已分配资源矩阵 Allocation 和最大需求矩阵 Max 如下表所示：

进程	Allocation (R1,R2,R3)	Max (R1,R2,R3)
P_1	(2, 1, 2)	(7, 5, 3)
P_2	(4, 0, 2)	(5, 2, 5)
P_3	(4, 0, 5)	(9, 0, 8)
P_4	(2, 0, 4)	(4, 2, 6)

   若当前系统剩余可利用资源向量 Available = (5, 4, 7)，关于当前系统的安全状态，下列分析正确的是？

   A. 系统处于不安全状态，必将发生死锁
   B. 系统处于安全状态，存在安全的执行序列，例如 <P_2, P_1, P_3 P_4,>
   C. 系统处于安全状态，唯一的安全序列是 <P_1, P_2, P_3, P_4>
   D. 系统处于不安全状态，因为当前剩余资源无法满足任何进程的最大需求

【答案】：B

【解析】：

先计算尚需资源矩阵 Need = Max - Allocation：

$Need_1$ = (7-2, 5-1, 3-2) = (5, 4, 1)

$Need_2$ = (5-4, 2-0, 5-2) = (1, 2, 3)

$Need_3$ = (9-4, 0-0, 8-5) = (5, 0, 3)

$Need_4$ = (4-2, 2-0, 6-4) = (2, 2, 2)

当前 Available = (5, 4, 7)

验证序列 <P_2, P_1, P_3 P_4,>：

检查 P_2：$Need_2$(1,2,3) ≤ Available(5,4,7)，满足。P_2 执行完释放，新 Available = (5+4, 4+0, 7+2) = (9, 4, 9)

检查 P_4：$Need_4$(2,2,2) ≤ (9,4,9)，满足。P_4 执行完释放，新 Available = (9+2, 4+0, 9+4) = (11, 4, 13)

检查 P_1：$Need_1$(5,4,1) ≤ (11,4,13)，满足。P_1 执行完释放，新 Available = (11+2, 4+1, 13+2) = (13, 5, 15)

检查 P_3：$Need_3$(5,0,3) ≤ (13,5,15)，满足

存在安全序列，因此系统处于安全状态

10. 【考点：死锁】操作系统通过破坏死锁的四个必要条件（互斥、占有并等待、不剥夺、循环等待）来进行死锁预防。下列预防措施与所破坏的必要条件匹配正确的是？

A. 采用 SPOOLing 假脱机技术将独占设备改造成共享设备 —— 破坏“不剥夺”条件

B. 规定所有进程必须一次性申请运行所需的全部资源，未满足前不投入运行 —— 破坏“占有并等待”条件

C. 对系统中的所有资源进行线性编号，要求进程严格按资源编号递增的顺序申请资源 —— 破坏“互斥”条件

D. 当进程申请新资源得不到满足时，强行释放其已占有的资源 —— 破坏“循环等待”条件

【答案】：B

【解析】：

A选项错误：SPOOLing 技术将独占设备改造为逻辑上的共享设备，破坏的是“互斥”条件

B选项正确：一次性申请全部资源（协议预分配），保证进程运行期间不再申请新资源，破坏的是“占有并等待（Hold and Wait）”条件

C选项错误：资源按序号递增申请（资源顺序分配法），破坏的是“循环等待”条件

D选项错误：强行剥夺已占有资源，破坏的是“不剥夺（No Preemption）”条件

11. 【考点：进程与线程的基本概念】在开发高并发的 Web 服务器（如类似 Nginx 或 Redis 的事件驱动模型）时，系统架构师需要深入权衡进程、线程模型以及进程间通信（IPC）机制的开销。关于操作系统中进程与线程的基本概念、实现模型及 IPC 机制，下列说法完全正确的是（ ）

A. 在纯用户级线程模型（多对一模型）中，线程的切换完全在用户态由线程库完成，开销极小。但如果其中一个用户级线程在执行同步 I/O 操作时因系统调用被阻塞，整个进程及其内部的所有其他线程都会被阻塞

B. 匿名管道（Pipe）是一种半双工的通信机制，在工程中常用于任意两个毫无关联的独立进程之间进行双向、高频的数据交换

C. 共享内存（Shared Memory）是所有 IPC 机制中速度最快的，因为它避免了数据在内核态和用户态之间的多次拷贝。并且，共享内存机制在操作系统底层已经内置了读写同步原语，开发者无需在代码中额外使用信号量进行加锁

D. 信号（Signal）机制（如 `SIGKILL`、`SIGTERM`）是一种基于共享缓存的同步通信方式，在工程中通常被用来在多个进程之间传递大规模的业务数据块

【答案】： A

【解析】：

A 选项完全正确：纯用户级线程（Many-to-One 模型）对操作系统内核是不可见的。内核只知道调度该进程，一旦进程中的某个线程发起了阻塞型的系统调用（如读文件、网络等待），内核就会

将整个进程挂起，导致该进程内的其他所有就绪线程都无法执行。这也是现代操作系统多采用内核级线程（一对一）或混合模型的原因

B 选项错误：匿名管道有两个致命限制：第一，通常是半双工的（单向数据流）；第二，只能用于具有亲缘关系（如父子进程、兄弟进程）的进程之间通信。无关联进程通信应使用命名管道（FIFO）

C 选项错误：共享内存确实是最快的 IPC，但操作系统并不提供针对共享内存的内置同步互斥机制。多个进程同时读写共享内存时，必须由开发者通过信号量（Semaphore）或互斥锁（Mutex）等机制自行维护同步，否则会导致数据竞争

D 选项错误：信号是一种异步通信机制（类似软件层面的中断），它仅仅传递一个极小的整型信号编号（通知事件发生），绝对不能用于传递“大规模的业务数据”

12. 【考点：CPU调度算法】在关系型数据库底层引擎中，大量并发事务的处理极度依赖操作系统的 CPU 调度策略与高效的上下文切换。关于 CPU 调度算法、多处理机调度以及上下文切换机制，下列说法完全正确的是（ ）

    A. 在多级反馈队列（MLFQ）调度算法中，为了防止 I/O 密集型进程发生饥饿，调度器通常会将频繁发生 I/O 阻塞的进程不断降级，最终放入优先级最低的队列中运行

    B. 进程上下文切换与线程上下文切换的核心性能差异在于：进程切换必须刷新 TLB（快表）并切换页表目录（如 CR3 寄存器）以更换虚拟地址空间；而同一进程内的线程切换无需切换虚拟地址空间，开销显著降低

    C. 在对称多处理机（SMP）调度中，设置“硬亲和性（Hard Affinity）”是指操作系统调度器必须频繁地将线程在多个不同的空闲 CPU 核心之间迁移，以达到全局负载均衡的最优解

    D. 在支持抢占式（Preemptive）调度的操作系统中，只要一个进程通过系统调用进入了内核态执行，在它返回用户态之前，无论发生什么高优先级中断，该进程都绝对不可被抢占

    【答案】： B

    【解析】：

    B 选项完全正确：这是工程实践中强烈推崇多线程而非多进程并发的核心原因。同一进程下的线程共享相同的虚拟地址空间。线程切换只涉及 CPU 寄存器（PC、SP 等）的保存与恢复；而进程切换由于虚拟地址空间改变，必须切换页表基址寄存器，这会导致 TLB（Translation Lookaside Buffer）失效，进而引发严重的内存访问性能下降

    A 选项错误：在 MLFQ 中，CPU 密集型进程（长时间占用时间片）会被降级到低优先级队列；而 I/O 密集型进程（在时间片用完前主动放弃 CPU）会被调度器认为交互性强，通常会被保留在高优先级队列，这恰恰是 MLFQ 兼顾交互性和吞吐量的设计精髓

    C 选项错误：处理器亲和性（Processor Affinity）是为了提高 CPU Cache 的命中率。“硬亲和性”是指强制将某个进程/线程绑定在指定的某个 CPU 核心上运行，禁止其迁移。频繁在核心间迁移会导致 Cache 刷新，降低性能

    D 选项错误：现代操作系统（如 Linux 2.6 以后的版本）支持内核态抢占（Kernel Preemption）。即使进程正处于内核态执行系统调用，只要它没有持有自旋锁等关键互斥资源，当更高优先级的进程就绪时，当前处于内核态的进程依然可以被抢占

13. 【考点：同步与互斥】在编写多线程同步控制组件（如各类锁机制）时，往往需要结合操作系统原语与底层硬件指令的支持。关于同步与互斥的实现方法、锁机制及条件变量，下列说法完全正确的是（ ）

    A. Test-and-Set（TSL）和 Compare-and-Swap（CAS）等操作是由操作系统内核通过软件算法（如 Peterson 算法）模拟实现的，它们无法真正在一条机器指令内完成

    B. 在单处理机（Uni-processor）且不开启抢占调度的系统中，使用自旋锁（Spinlock）来保护临界区是非常高效的，因为自旋锁避免了线程阻塞和唤醒的上下文切换开销

    C. 互斥锁（Mutex）与二元信号量（Binary Semaphore）在语义上完全等价。在工程实现中，任何一个线程都可以随时调用 `V()` 操作去释放一个由其他线程加锁的互斥锁

    D. 在管程（Monitor）机制中，当线程由于条件不满足而调用条件变量的 `wait()` 操作被阻塞时，该操作在底层会自动释放当前线程持有的管程互斥锁，以允许其他线程进入管程

【答案】： D
【解析】：
D 选项完全正确：这是管程和条件变量设计的核心逻辑（如 Java 的 `Object.wait()` 或 pthread 的 `pthread_cond_wait()`）。如果 `wait()` 不释放管程互斥锁，那么当前线程在里面睡觉，其他线程又因为锁被占用无法进入管程去改变条件并执行 `signal()`/`notify()`，系统将陷入永久死锁
A 选项错误：TSL 和 CAS 是硬件直接提供的原子指令，依靠锁总线或缓存一致性协议在物理硬件层面保证原子性，绝非软件算法模拟
B 选项错误：在单处理机且不可抢占的系统中，自旋锁是灾难性的。获取不到锁的线程会执行死循环（自旋），霸占着唯一的 CPU；而持有锁的线程得不到 CPU 去释放锁，必然导致系统死锁（或者被看门狗复位）。自旋锁只适用于多处理机环境且临界区极短的场景
C 选项错误：互斥锁（Mutex）具有所有权（Ownership）概念，只有加锁的那个线程才能进行解锁操作。而信号量（Semaphore）没有所有权概念，一个线程执行 `P()` 阻塞后，完全可以由另一个不相关的线程执行 `V()` 来唤醒它

14. 【考点：经典同步问题】经典同步问题是现代软件工程中各种消息队列、缓存池设计的理论抽象。在生产者-消费者问题、读者-写者问题等经典模型中，稍微不当的 P、V 操作顺序就会导致灾难性后果。下列关于经典同步问题的论述，完全正确的是（ ）

    A. 在解决生产者-消费者问题时，如果缓冲区大小为 N，为了防止生产者覆盖未消费的数据，必须在执行 `P(empty)`（申请空闲缓冲块）之前先执行 `P(mutex)`（申请互斥锁），以确保申请过程的原子性
    B. 在经典的“读者优先（Reader-Preference）”的读者-写者问题解法中，只要系统中源源不断地有新的读者进程到达并开始读取，等待写入的写者进程就会永远处于阻塞状态，即发生“写者饥饿”
    C. 哲学家进餐问题的最优解是：规定所有哲学家都必须先拿起左手的筷子，然后再尝试拿起右手的筷子。这种策略既能保证最大并发度，又能从根本上避免死锁
    D. 管程（Monitor）将共享数据和同步操作封装在一起，因此程序员在使用管程时，仍然需要手动在每个函数的开头和结尾显式地编写 P 和 V 操作来实现对共享数据的互斥访问

    【答案】： B
    【解析】：
    B 选项完全正确：经典的读者优先算法中，第一个读者到来会锁住写者（`P(rw_mutex)`），后续读者到来只要发现当前有读者正在读（`read_count > 0`），就可以直接进入临界区而不需要再次申请 `rw_mutex`。这意味着只要读者不断到来，`read_count` 就不会降为 0，锁就不会释放，写者会无限期饥饿
    A 选项错误：这是极其经典的死锁陷阱。如果先执行 `P(mutex)` 拿到互斥锁，接着执行 `P(empty)`。若此时缓冲区已满，生产者会在 `P(empty)` 处阻塞，但它此时并未释放 `mutex`。消费者想要去消费数据执行 `V(empty)`，却在第一步就被 `P(mutex)` 挡在门外。两者互相等待，发生死锁。必须先申请资源信号量，再申请互斥信号量
    C 选项错误：所有哲学家同时拿起左手筷子，系统会进入死锁（每个人都等右边的人放下筷子）。这不仅不能避免死锁，反而是产生“环路等待”从而导致死锁的经典反例
    D 选项错误：管程的互斥特性是由编译器或编程语言层面（如 Java 的 `synchronized` 关键字）自动保证的。每次只有一个进程能进入管程，程序员不需要也不应该在管程的方法内部再手动编写互斥的 P/V 操作

15. 【考点：死锁】在复杂的资源分配系统（如分布式数据库锁管理器）中，如果不加以严格的约束，很容易陷入多个事务相互等待资源的死锁状态。关于死锁的四个必要条件、死锁预防、死锁避免以及死锁检测，下列说法完全正确的是（ ）

    A. 为了破坏死锁的“请求和保持（Hold and Wait）”条件，系统可以要求每个进程在运行前一次性申请并分配其所需的全部资源。这种死锁预防策略在工程中被广泛采用，因为它能极大提高系统整体的资源利用率
    B. 银行家算法（Banker's Algorithm）属于死锁检测（Deadlock Detection）算法。它通过周期性地扫描系统资源分配图，利用深度优先搜索（DFS）寻找图中是否存在环路，从而判断系统是否已经死锁

C. 破坏死锁的“循环等待（Circular Wait）”条件的一种经典实现是：对系统中的所有资源类型进行全局线性编号，并强制规定每个进程只能按编号递增的顺序去申请资源

D. 如果在使用银行家算法的系统中，发现当前系统状态是不安全状态（Unsafe State），则意味着系统已经发生了死锁，必须立即强制中止部分进程以解除死锁

【答案】： C

【解析】：

C 选项完全正确：这是死锁预防（Deadlock Prevention）中破坏“循环等待”条件的标准做法（即“顺序资源分配法”）。因为要求资源只能按递增顺序申请，所以绝不可能出现进程 A 拿着高编号资源等低编号资源，而进程 B 拿着低编号资源等高编号资源的情况，从而物理上阻断了环路等待的发生

A 选项错误：一次性全部分配资源确实能破坏“请求和保持”条件（即死锁预防中的“静态分配”），但它的致命弱点是会导致极其低下的资源利用率（进程可能长时间霸占近期不用的资源）并且容易导致进程饥饿。因此在工程中通常不作为首选策略

B 选项错误：银行家算法属于死锁避免（Deadlock Avoidance），不是死锁检测。它是在资源分配之前，通过模拟分配并检查系统是否能找到一个安全序列（Safe Sequence）来决定是否真正分配资源。扫描分配图找环路是死锁检测（Deadlock Detection）的方法，属于死锁发生后的补救措施

D 选项错误：“不安全状态”和“死锁”不是等价概念。死锁状态一定是不安全状态，但不安全状态不一定是死锁状态。不安全状态仅仅意味着系统如果继续按照最坏情况分配资源，*可能*会走向死锁，但如果某些进程提前归还了资源，死锁实际上未必发生

16. **72**．A

当操作系统终止进程时，所有的进程资源（如内存空间、进程控制块、设备、打开文件、I/O 缓冲区等）都会被释放。有些系统不允许子进程在父进程已终止的情况下存在，对于这类系统，若一个进程终止，则它的所有子进程也终止，这种现象称为级联终止。但是，不是所有操作系统都是这么设计的，因此终止子进程不一定在终止进程时执行。

17. **73**．B

线程可理解为轻量级进程，仅拥有一点必不可少、能保证独立运行的资源。例如，在每个线程中都有一个用于控制线程运行的线程控制块（TCB）、用于指示被执行指令序列的程序计数器、保留局部变量、少数状态参数和返回地址等的一组寄存器和堆栈。因此，线程 T 的栈空间是线程 T 所独有的，不会被线程 Ta 和 Tb 共享。多个线程共享所属进程所拥有的资源，进程 P 的线程可以访问进程 P 的全部地址空间和资源（如已打开的文件、定时器、信号量机构等）以及它申请到的 I/O 设备等。综上所述，Ta 和 Tb 可共享的是进程 P 的地址空间和文件描述符 fd。

18. **54**．C

系统使用时间片轮转调度算法，最初共有 10 个进程处于就绪队列，进程 P 处于队尾，时间片大小为 5ms。假设当 $t=0$ 时开始进程调度，就绪队列的前 9 个进程依次运行一个时间片，则当 $t=45$ms 时调度进程 P 运行，运行 5ms 后，当 $t=50$ms 时又开始新一轮循环。进程 P 所需的 CPU 时间为 25ms，共需 5 轮循环，其他进程所需的 CPU 时间都比进程 P 的长，这 5 轮循环中其余 9 个进程都不会运行结束，因此进程 P 的周转时间为 $50\times5=250$ms。

19. **55**．D

进程切换时，需要进行上下文切换，主要完成：① 将当前 CPU 中各寄存器的内容保存到当前进程的 PCB 中；② 将新进程的现场信息装入 CPU 的各个寄存器。③ 将控制转至新进程执行，即更新程序计数器（PC）的值。栈基址寄存器用来指明系统内核栈位置的寄存器，页表基址寄存器用来指明当前进程顶级页表基地址的寄存器，属于进程控制信息。因此，说法 I、II、III 均需要更新。

20. **56**．C

在该调度系统中，就绪队列采用按优先权从高到低排序的有序单链表实现。插入新进程时，为维持链表的有序性，需要从表头开始顺序查找，直至找到首个优先权不高于待插入进程的结点，并在其前插入；最坏情况下需要遍历整个链表，时间复杂度为 $O(n)$。由于最高优先权的进程始终位于链表头部，调度程序只需访问头结点即可获得下一个待执行的进程，无须额外搜索，因此选择进程的操作时间复杂度为 $O(1)$。

21.

57．D

进程上下文切换需要**隔离各进程的执行环境**，并确保新进程正确恢复运行，因此必须更新**当前进程的上下文**。通用寄存器和程序计数器属于典型的进程私有状态：前者保存临时数据，后者指向下一条指令地址，两者都必须在切换时保存和恢复。页表基址寄存器存放当前进程页表的物理起始地址。在页式系统中，每个进程都拥有独立的虚拟地址空间和页表，切换时必须更新该寄存器，以激活新进程的地址映射。内核中断向量表基址寄存器指向操作系统内核维护的中断处理入口表，该表是全局共享的系统资源，用于响应中断和异常，其内容由内核统一管理，不随用户进程变化，因此在进程上下文切换时无须更新。

22. **01**．【解答】

1）进程控制块（PCB）是操作系统内核用于管理进程的核心数据结构，始终位于**内核区**。当进程执行 scanf()等待键盘输入时，因需等待 I/O 完成而主动放弃 CPU，进入**阻塞态**。

2）main()函数的代码存储在**只读代码段**中。scanf()和 printf()虽为标准库函数，但其底层通过系统调用访问设备（如键盘、终端），最终依赖**设备驱动程序**完成实际 I/O 操作。

3）ptr 是全局变量，被分配在**可读/写数据段**；length 是 main()中的局部变量，若未被优化至寄存器，则分配在**用户栈**；ptr 指向的字符串由 malloc()动态分配，位于**运行时堆**中。

23. **10**．【解答】

1）因为采用了静态优先数，当就绪队列中总有优先数较小的进程时，优先数较大的进程一直没有机会运行，所以会出现饥饿现象。

2）一种动态优先数计算方法为 $\text{priority} = \text{nice} + k_1 \times \text{cpuTime} - k_2 \times \text{waitTime}$，其中 $k_1 > 0$，$k_2 > 0$，分别用来调整 cpuTime 和 waitTime 在 priority 中所占的比例。若一个进程的运行时间较长，则其 cpuTime 就增加，进而降低其优先级；若一个进程的等待时间较长，则其 waitTime 增加，进而会提高其优先级。于是，waitTime 就可使长时间等待的进程优先数减少，进而避免出现饥饿现象。

24.

17.【解答】

互斥资源：缓冲区只能互斥访问，因此设置互斥信号量 mutex。

同步问题：P_1, P_2 因为奇数的放置与取用而同步，设同步信号量 odd；P_1, P_3 因为偶数的放置与取用而同步，设置同步信号量 even；P_1, P_2, P_3 因为共享缓冲区，设同步信号量 empty，初值为 N。程序如下：

```
semaphore mutex=1;              //缓冲区操作互斥信号量
semaphore odd=0,even=0;         //奇数、偶数进程的同步信号量
semaphore empty=N;              //空缓冲区单元个数信号量
coBegin{
    Process P1()
    while(True)
    {
        x=produce();            //生成一个数
        P(empty);               //判断缓冲区是否有空单元
        P(mutex);               //缓冲区是否被占用
        Put();
        V(mutex);               //释放缓冲区
        if(x%2==0)
            V(even);            //若是偶数，向 P3 发出信号
        else
            V(odd);             //若是奇数，向 P2 发出信号
    }
    Process P2()
    while(True)
    {
        P(odd);                 //收到 P1 发来的信号，已产生一个奇数
        P(mutex);               //缓冲区是否被占用
        getodd();
        V(mutex);               //释放缓冲区
        V(empty);               //向 P1 发信号，多出一个空单元
        countodd();
    }
    Process P3()
    while(True)
    {
        P(even);                //收到 P1 发来的信号，已产生一个偶数
        P(mutex);               //缓冲区是否被占用
        geteven();
        V(mutex);               //释放缓冲区
        V(empty);               //向 P1 发信号，多出一个空单元
        counteven();
    }
}coEnd
```

25.

18.【解答】

互斥资源：取号机（一次只有一位顾客取号），因此设置互斥信号量 mutex。

同步问题：顾客需要获得空座位等待叫号。营业员空闲时，将选取一位顾客并为其服务。空座位的有无会影响等待顾客的数量，顾客的有无决定了营业员是否能开始服务，因此分别设置信号量 empty 和 full 来实现这一同步关系。另外，顾客获得空座位后，需要等待叫号和被服务。这样，顾客与营业员就服务何时开始又构成了一个同步关系，定义信号量 service 来完成这一同步过程。

```
semaphore empty=10;          //空座位的数量，初值为 10
semaphore mutex=1;           //互斥使用取号机
semaphore full=0;            //已占座位的数量，初值 0
semaphore service=0;         //等待叫号
coBegin
{
    Process 顾客 i
    {
        P(empty);            //等空位
        P(mutex);            //申请使用取号机
        从取号机上取号;
        V(mutex);            //取号完毕
        V(full);             //通知营业员有新顾客
        P(service);          //等待营业员叫号
        接受服务;
    }
    Process 营业员
    {
        while(True){
            P(full);         //没有顾客则休息
            V(empty);        //离开座位
            V(service);      //叫号
            为顾客服务;
        }
    }
}coEnd
```

26. **19.【解答】**

出入口一次仅允许一人通过，设置互斥信号量 mutex，初值为 1。博物馆最多可以同时容纳 500 人，因此设置信号量 empty，初值为 500。

```
semaphore empty=500;         //博物馆可以容纳的最多人数
semaphore mutex=1;           //用于出入口资源的控制
coBegin
参观者进程 i:
{
    …
    P(empty);                //可容纳人数减 1
    P(mutex);                //互斥使用门
    进门;
    V(mutex);
    参观;
    P(mutex);                //互斥使用门
    出门;
    V(mutex);
    V(empty);                //可容纳人数增 1
    …
}
coEnd
```

27.

20.【解答】

这是典型的生产者和消费者问题，只对典型问题加了一个条件，只需在标准模型上新加一个信号量，即可完成指定要求。

设置 4 个变量 mutex1, mutex2, empty 和 full，mutex1 用于控制一个消费者进程在一个周期（10 次）内对缓冲区的访问，初值为 1；mutex2 用于控制进程单次互斥地访问缓冲区，初值为 1；empty 代表缓冲区的空位数，初值为 1000；full 代表缓冲区的产品数，初值为 0。伪代码如下：

```
semaphore empty=1000;     //空缓冲区数
semaphore full=0;         //非空缓冲区数
semaphore mutex1=1;       //用于实现生产者之间的互斥
semaphore mutex2=1;       //用于实现消费者之间的互斥
int in=0;
int out=0;
```

生产者进程

```
while(TRUE){
    produce;
    P(empty);
    P(mutex1);
    put an item into buf[in];
    in=(in+1) mod n;
    V(mutex1);
    V(full);
}
```

消费者进程

```
while(TRUE){
    P(mutex2);
    for(int i=0;i<10;i++){
        P(full);
        get an item from buf[out];
        out=(out+1) mod n;
        V(empty);
    }
    V(mutex2);
    Consume;
}
```

28. **21.【解答】**

本题是一个典型的生产者-消费者问题。A 和 B 既是生产者，又是消费者。首先分析题中的互斥关系，信箱 A 和信箱 B 作为生产者-消费者问题中的缓冲区，需要互斥访问，因此设置两个信号量 mutex_A 和 mutex_B。然后分析 A 和 B 的同步关系，A 有两个动作，分别是从信箱 A 中取出邮件和将新邮件放入信箱 B，因此设置信号量 Full_A 和 Empty_B，用来保证信箱 A 中有邮件可以取，信箱 B 中有空间可以放。B 的分析同理。因此本题的所有信号量设置如下：

```
semaphore Full_A = x;       // Full_A 表示 A 的信箱中的邮件数量
semaphore Empty_A = M-x;    // Empty_A 表示 A 的信箱中还可存放的邮件数量
semaphore Full_B = y;       // Full_B 表示 B 的信箱中的邮件数量
semaphore Empty_B = N-y;    // Empty_B 表示 B 的信箱中还可存放的邮件数量
semaphore mutex_A = 1;      // mutex_A 用于 A 的信箱互斥
semaphore mutex_B = 1;      // mutex_B 用于 B 的信箱互斥
```

互斥信号量的 PV 操作直接放置在对信箱操作的前后。在 A 取邮件之前需执行 P(Full_A)来检查信箱 A 中是否有邮件可取，取出邮件之后需执行 V(Empty_A)，表示新增了一个邮件大小的空间；在 A 将邮件放入邮箱 B 之前需执行 P(Empty_B)来检车信箱 B 中是否有空间可以放，放完邮

件之后需执行 V(Full_B)，表示信箱 B 增加了一个邮件。B 的分析同理。

```
coBegin
```

A{	B{

while(TRUE){
P(Full_A);
P(mutex_A);
从 A 的信箱中取出一个邮件;
V(mutex_A);
V(Empty_A);
回答问题并提出一个新问题;
P(Empty_B);
P(mutex_B);
将新邮件放入 B 的信箱;
V(mutex_B);
V(Full_B);
}
	while(TRUE){
	P(Full_B);
	P(mutex_B);
	从 B 的信箱中取出一个邮件;
	V(mutex_B);
	V(Empty_B);
	回答问题并提出一个新问题;
	P(Empty_A);
	P(mutex_A);
	将新邮件放入 A 的信箱;
	V(mutex_A);
	V(Full_A);
	}

}	}

```
coEnd
```

29. **22.【解答】**

对于这类问题，做法如下：

首先，找出有可能需要互斥访问的变量，只有全局变量才可能需要互斥访问，也就是说，含全局变量的代码段才可能是临界区。而局部变量是不需要互斥访问的，就算不同的进程之间都有相同名字的局部变量，但对不同的进程来说，它们只认识属于自己的局部变量。

其次，有关互斥的难点不在于找出哪些变量有可能需要互斥访问，而要理解互斥是进程之间对某个变量的互斥，不能脱离具体的进程来谈某个变量需不需要互斥访问。前面说了全局变量有可能需要互斥访问，但也不一定，因为这要看进程对它的操作，因此，一组互斥关系需要指出是哪些进程对哪个变量的互斥。例如，考虑两种情况：第一种情况是，进程 1 和进程 2 需要对共享变量 C 互斥访问，进程 1 和进程 3 也需要对变量 C 互斥访问，这是两组不同的互斥关系，而不能简单地理解为对 C 的互斥；第二种情况是，进程 1、2、3 两两之间都需要对变量 C 互斥访问。这种情况与第一种情况的差别是，第一种情况允许进程 2、3 对变量 C 的访问是不互斥的。若题目给出的是第一种情况，而我们采用了第二种情况去解题，则可能有同学认为，我依然满足了题目的要求啊，进程 1、2 之间以及进程 1、3 之间对 C 的访问都是互斥的，但这样降低了并发度，因为第二种情况允许进程 2、3 对变量 C 的访问是不互斥的，这也是 PV 互斥的最大难点，就是在没有准确找准互斥关系的条件下，容易导致程序的并发度降低，从而扣分。

本题的分析如下。

首先，全局变量是 x、y、z，只有对这三个变量的访问才可能需要互斥。线程 1 涉及 x、y 的访问（只读），线程 2 涉及 y、z 的访问（只读），线程 3 涉及 y、z 的访问（读和写）。

其次，找互斥关系，根据什么原则呢？答案是读者-写者原则。读和读之间不需要互斥，读和写之间、写和写之间需要互斥。因此，本题的互斥关系如下：第一对，thread1 和 thread3 之间需要对变量 y 的访问互斥；第二对，线程 2 和线程 3 之间需要对变量 y 的访问互斥；第三对，线程 2 和线程 3 之间需要对变量 z 访问的互斥。正确找出所有互斥关系后，接下来的操作就很简单。因为有三组互斥关系，所以定义三个互斥信号量，分别为 mutex_y1 = 1、mutex_y2 = 1 和 mutex_z = 1，然后分别将它们加到线程代码段的相应位置。例如，在线程 1 的 w = add(x, y)和线程 3 的 y = add(y, w)

上下用 mutex_y1 夹住；在线程 2 的 w = add(y, z)和线程 3 的 z = add(z, w)上下用 mutex_y2 夹住；在线程 2 的 w = add(y, z)和线程 3 的 y = add(y, w)上下用 mutex_z 夹住。

```
semaphore mutex_y1=1; //mutex_y1 用于 thread1 与 thread3 对变量 y 的互斥访问
semaphore mutex_y2=1; //mutex_y2 用于 thread2 与 thread3 对变量 y 的互斥访问
semaphore mutex_z=1;  //mutex_z 用于 thread2 与 thread3 对变量 z 的互斥访问
```

互斥代码如下：

thread1	thread2	thread3
{	{	{
cnum w;	cnum w;	cnum w;
wait(mutex_y1);	wait(mutex_y2);	w.a = 1;
w = add(x, y);	wait(mutex_z);	w.b = 1;
signal(mutex_y1);	w = add(y, z);	wait(mutex_z);
…	signal(mutex_z);	z = add(z, w);
}	signal(mutex_y2);	signal(mutex_z);
	…	wait(mutex_y1);
	}	wait(mutex_y2);
		y = add(y, w);
		signal(mutex_y1);
		signal(mutex_y2);
		…
		}

30. **23.【解答】**

回顾传统的哲学家问题，假设餐桌上有 n 名哲学家、n 根筷子，那么可以用这种方法避免死锁（本书考点讲解中提供了这一思路）：限制至多允许 $n-1$ 名哲学家同时“抢”筷子，那么至少会有 1 名哲学家可以获得两根筷子并顺利进餐，于是不可能发生死锁的情况。

本题可以用碗这个限制资源来避免死锁：当碗的数量 m 小于哲学家的数量 n 时，可以直接让碗的资源量等于 m，确保不会出现所有哲学家都拿一侧筷子而无限等待另一侧筷子进而造成死锁的情况；当碗的数量 m 大于或等于哲学家的数量 n 时，为了让碗起到同样的限制效果，我们让碗的资源量等于 $n-1$，这样就能保证最多只有 $n-1$ 名哲学家同时进餐，所以得到碗的资源量为 $\min\{n-1, m\}$。在进行 PV 操作时，碗的资源量起限制哲学家取筷子的作用，所以需要先对碗的资源量进行 P 操作。具体过程如下：

```
//信号量
semaphore bowl;                 //用于协调哲学家对碗的使用
semaphore chopsticks[n];        //用于协调哲学家对筷子的使用
for(int i=0;i<n;i++)
    chopsticks[i]=1;            //设置两名哲学家之间筷子的数量
bowl=min(n-1,m);                //bowl≤n-1，确保不死锁

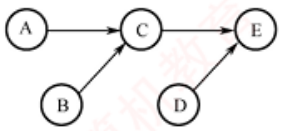
coBegin
while(TRUE){                    //哲学家 i 的程序
    思考;
    P(bowl);                    //取碗
    P(chopsticks[i]);           //取左边筷子
    P(chopsticks[(i+1)%n]);     //取右边筷子
    就餐;
    V(chopsticks[i]);
    V(chopsticks[(i+1)%n]);

    V(bowl);
}
coEnd
```

31.

24.【解答】

本题是一个典型的利用信号量实现前驱关系的同步问题。首先画出各个操作之间的执行顺序图，可以看出，A、B、D 的执行不需要任何前提条件。执行完 A 和 B 之后才能执行 C，存在两对同步关系：A→C 和 B→C，设置两个同步变量 SAC＝0 和 SBC＝0，完成 A 和 B 之后分别执行 V(SAC)和 V(SBC)，表示 A 或 B 已完成；执行 C 之前需要执行 P(SAC)和 P(SBC)，检查 A 和 B 是否完成。执行完 C 和 D 之后才能执行 E，也存在两对同步关系：C→E 和 D→E，因此再设置两个同步变量 SCE＝0 和 SDE＝0，完成 C 和 D 之后分别执行 V(SCE)和 V(SDE)，表示 C 或 D 已完成；执行 E 之前需要执行 P(SCE)和 P(SDE)，检查 C 和 D 是否完成。




```
semaphore SAC = 0;        //控制 A 和 C 的执行顺序
semaphore SBC = 0;        //控制 B 和 C 的执行顺序
semaphore SCE = 0;        //控制 C 和 E 的执行顺序
semaphore SDE = 0;        //控制 D 和 E 的执行顺序
```

5 个操作可描述为如下。

```
coBegin
A(){
    完成动作 A;
    V(SAC);              //实现 A、C 之间的同步关系
}
B(){
    完成动作 B;
    V(SBC);              //实现 B、C 之间的同步关系
}
C(){
    //C 必须在 A、B 都完成后才能完成
    P(SAC);
    P(SBC);
    完成动作 C;
    V(SCE);              //实现 C、E 之间的同步关系
}
D(){
    完成动作 D;
    V(SDE);              //实现 D、E 之间的同步关系
}
E(){
    //E 必须在完成 C、D 之后执行
    P(SCE);
    P(SDE)
    完成动作 E;
}
coEnd
```

32. **25.【解答】**

1）信号量 S 是能被多个进程共享的变量，多个进程都可通过 wait()和 signal()对 S 进行读、写操作。所以，wait()和 signal()操作中对 S 的访问必须是互斥的。

2）方法 1 错误。在 wait()中，当 S <= 0 时，关中断后，其他进程无法修改 S 的值，while 语句陷入死循环。方法 2 正确。方法 2 在循环体中有一个开中断操作，这样就可以使其他进程修改 S 的值，从而避免 while 语句陷入死循环。

3）用户程序不能使用开/关中断指令实现临界区互斥。因为开中断和关中断指令都是特权指令，不能在用户态下执行，只能在内核态下执行。

33.

26.【解答】

本题是一个典型的利用信号量实现前驱关系的同步问题。需要强调的是，只有不同进程之间的操作才需要进行同步。进程 T1 依次执行 A、E、F，进程 T2 依次执行 B、C、D。我们需要分析哪些操作必须在另一个进程的某个操作完成之后才能执行。由图可知，对进程 T1 来说，E 必须在进程 T2 执行完 C 后才能执行；对进程 T2 来说，C 必须在进程 T1 执行完 A 后才能执行。因此，有两对同步关系：A→C 和 C→E。为了实现这两对同步关系，定义两个同步信号量 S_{AC} 和 S_{CE}。进程 T1 执行完 A 后，发出信号 signal(S_{AC})，表示 A 已执行完成；进程 T2 准备执行 C 之前，等待信号 wait(S_{AC})，检查 A 是否执行完成。同理，进程 T2 执行完 C 后，发出信号 signal(S_{CE})，表示 C 已执行完成；进程 T1 准备执行 E 之前，等待信号 wait(S_{CE})，检查 C 是否执行完成。这样就保证了两个进程之间的同步。

semaphore $S_{AC}=0$; //描述 A、C 之间的同步关系
semaphore $S_{CE}=0$; //描述 C、E 之间的同步关系	

T1:
A;
signal(S_{AC});
wait(S_{CE});
E;
F;
	T2:
	B;
	wait(S_{AC});
	C;
	signal(S_{CE});
	D;

34. **27.【解答】**

1）if 语句无法实现对临界区的互斥访问，因为 if 语句执行后，不论结果如何，线程都能访问临界区。本题使用 swap 指令和 lock 变量来实现对临界区的互斥访问，当线程不能进入临界区时，本身并不会主动放弃 CPU，因此需要让线程在进入区中循环检查 lock 值，可以使用 while 循环，当 lock 值为 TRUE 时，线程一直执行 while 循环的内容，直到 lock 值被修改为 FALSE 时，线程才能进入临界区，因此将进入区中的语句“if (key == TRUE) swap key, lock”修改为“while (key == TRUE) swap key, lock”。在退出区中，代表该线程对临界资源的访问已经结束，此时需要将 lock 值置为 FALSE，代表其他线程可以访问临界区，因此将退出区中的语句“lock = TRUE”修改为“lock = FALSE”。

2）否。因为多个线程可以并发执行 newSwap()，newSwap()执行时传递给形参 b 的是共享变量 lock 的地址，在 newSwap()中对 lock 既有读操作又有写操作，并发执行时不能保证实现两个变量值的原子交换，从而导致并发执行的线程同时进入临界区。例如，线程 A 和线程 B 并发执行，初始时 lock 值为 FALSE，当线程 A 执行完*a = *b 后发生了进程调度，切换到线程 B 执行，线程 B 执行完 newSwap 后发生线程切换，此时线程 A 和 B 都能进入临界区，不能实现互斥访问。

35. **28.【解答】**

1）代码 C_1 执行对 B 的写操作，且 P_1 和 P_2 需要互斥执行 C_1，因此 C_1 的代码是临界区。

2）有一组互斥关系和一组同步关系。互斥关系为：P_1 和 P_2 需要互斥访问缓冲区 B；同步关系为：P_1 执行 C_1 后，P_2 才能执行 C_2。缓冲区 B 初始为空，且最大容量为 1，因此可不用定义互斥信号量 mutex。因为在缓冲区大小为 1 的条件下，同步信号量 S 就可同时保证 P_1 和 P_2 互斥访问 B，符合题意“尽可能定义少的信号量”的要求。进程 P_1 和 P_2 的同步伪代码如下。

Semaphore S = 0；//实现进程 P1 与 P2 的同步	
P1 … C1; signal(S); …	P2 … wait(S); C2; …

3）缓冲区 B 初始非空，因此仅需一个互斥信号量 mutex 来保证 P_1 和 P_2 互斥访问 B 即可。进程 P_1 和 P_2 的互斥伪代码如下。

Semaphore mutex = 1；//实现 P1 与 P2 互斥执行 C3	

P1
…
wait(mutex);
C3;
signal(mutex);
…
	P2
	…
	wait(mutex);
	C3;
	signal(mutex);
	…

36. **29.【解答】**

本题需协调甲、乙、丙三人的协作过程，并管理共享资源。水桶仅由丙使用，无竞争；铁锹由甲（挖树坑）和乙（填土）共用，需互斥访问，设置一个互斥信号量 **mutex**。本题有三组同步关系：① 未被使用的树坑数量必须小于 3，甲才可挖新坑；② 乙必须在有树坑时才能放树苗并填土；③ 丙必须在有已填土的树苗时才能浇水。为此，还需定义三个同步信号量：**position** 表示还可挖的树坑配额（允许新增的未使用树坑数量），初值为 3；**pit** 表示当前已挖好但尚未被使用的树坑数量，初值为 0；**tree** 表示已完成填土、等待浇水的树苗数量，初值为 0。

三人操作流程如下：甲先执行 wait(position)申请一个树坑配额，成功后获取铁锹挖坑，释放铁锹后执行 signal(pit)，通知乙有新树坑可用；乙先执行 wait(pit)等待树坑，放树苗后立即执行 signal(position)释放一个树坑配额（因可用树坑减少 1 个），随后在 mutex 保护下使用铁锹填土，完成后执行 signal(tree)通知丙；丙执行 wait(tree)等待可浇水的树苗，然后浇水。

Semaphore position = 3; //可挖树坑的数量
Semaphore pit = 0; //树坑的数量
Semaphore tree = 0; //待浇水的树苗数
Semaphore mutex = 1; //实现铁锹的互斥使用		

甲：
for(;;)
{
wait(position);
wait(mutex);
挖树坑;
signal(mutex);
signal(pit);
…
}
	乙：
	for(;;)
	{
	wait(pit);
	放树苗;
	signal(position);
	wait(mutex);
	填土;
	signal(mutex);
	signal(tree);
	…
	}
		丙：
		for(;;)
		{
		wait(tree);
		浇水;
		…
		}